Unit 1 - Math Fundamentals
1.5 Floating-Point Arithmetic

Graham West

MSDS 520: Math and Stats Foundations for Data Science
Meharry Medical College

Outline

1. Why Finite Precision?
2. How a Machine Stores a Number
3. Where It Bites, and Why It Usually Does Not
4. Review

Why Finite Precision?

A Problem to Motivate Us

Type this into Python

```
>>> 0.1 + 0.2 == 0.3   →   False
>>> 0.1 + 0.2   →   0.30000000000000004
```

Not a bug, and not a Python quirk: R, C, Java and Excel all agree. The machine cannot store 0.1 exactly, for the same reason you cannot write $\frac{1}{3} = 0.333\ldots$ in full: you write 0.333 and stop, and have quietly written down a number that is *not* a third.

A computer has finite memory

A number is stored in a fixed number of **bits**, 64 typically, so only **finitely many** numbers exist exactly. Every other real number is replaced by the nearest one the machine has. That substitution is **rounding**, and the difference is **rounding error**.

Almost always this is harmless: the error is about one part in 10^{16}, and no measurement you will ever feed a computer is good to one part in 10^{3}. This lecture is about the handful of *shapes* of computation where that tiny error gets amplified, because those arrive **silently**.

Where This Shows Up in Data Science

Ordinary data-science arithmetic, means, regressions, distances, is fine: the rounding is far below the noise in the data. The three situations worth recognizing all have a recognizable **shape**:

The situations to recognize

- **A sum spanning many magnitudes.** The harmonic series $\sum 1/n$ is unbounded, yet summed in a loop the total freezes; a million small terms added to a large total vanish.
- **A long product.** Multiplying many probabilities collapses to exactly 0, whose log is $-\infty$.
- **Subtracting nearly equal numbers.** A naive variance formula can come out *negative*.

None of these raise an error, which is why the shapes are worth knowing on sight. Each has a one-line fix, and the lecture ends with the list.

What We Will Build

The plan

1. **Binary**, then the **floating-point system** (β, p, L, U): how a machine writes a number down, how many numbers it can write, and the four numbers that describe it, including **machine epsilon**.
2. The few places it **bites**: **absorption**, **overflow** and **underflow**, and **cancellation**, each with the habit that avoids it, and why everything else is safe to ignore.

This is an **overview**. The aim is that you recognize these failures when they bite, not that you can derive the standard.

Notebook for this lecture: `U1-5_FloatingPoint-1_MachineEpsilon`.

How a Machine Stores a Number

Place Value: Just Add Up the Places

A digit's **position** says which power of the base it multiplies: 0 at the units place, up to the left, *negative* to the right of the point. In base 10 you do this without thinking:

Decimal: 253.75_{10}

$$\begin{aligned} 253.75_{10} &= 2 \cdot 10^2 + 5 \cdot 10^1 + 3 \cdot 10^0 + 7 \cdot 10^{-1} + 5 \cdot 10^{-2} \\ &= 200 + 50 + 3 + 0.7 + 0.05 \end{aligned}$$

Binary: 11001.101_2, the same scheme with digits 0 and 1

$$\begin{aligned} 11001.101_2 &= 1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 + 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} \\ &= 16 + 8 + 1 + \tfrac{1}{2} + \tfrac{1}{8} \;=\; 25.625_{10} \end{aligned}$$

Two digits, because a transistor is reliably off or on, so the places are powers of 2. Note the fractions available: **a binary fraction is a sum of powers of one half**, halves, quarters, eighths, and nothing else. That single fact is why 0.1 is not exact.

Scientific Notation, in Any Base

Scientists write a wide-ranging number as a modest number times a power of ten:

$$25.625 \;=\; \underbrace{+}_{\text{sign}}\; \underbrace{2.5625}_{\text{mantissa}} \times \underbrace{10}_{\text{base}}\overbrace{{}^{1}}^{\text{exponent}}$$

A computer does exactly the same thing in base 2:

$$11001.101_2 \;=\; +\,1.1001101_2 \times 2^4$$

Why "floating" point

The decimal/binary point does not sit in a fixed place, the exponent **floats** it left or right. That is what lets 64 bits cover everything from the mass of a proton to the mass of a galaxy, instead of a narrow fixed range.

The Formal Definition

A floating-point system

Every representable number has the form

$$x = \pm \left(d_0 + \frac{d_1}{\beta} + \frac{d_2}{\beta^2} + \cdots + \frac{d_{p-1}}{\beta^{p-1}} \right) \beta^E$$

where

- β is the **base** (2 for a computer, 10 for us);
- p is the **precision**, the number of digits kept;
- $d_0, d_1, \ldots, d_{p-1}$ are those digits, the **mantissa**, each with $0 \le d_i \le \beta - 1$;
- E is the **exponent**, an integer restricted to a range $L \le E \le U$.

Four integers, β, p, L, U, therefore define the whole system.

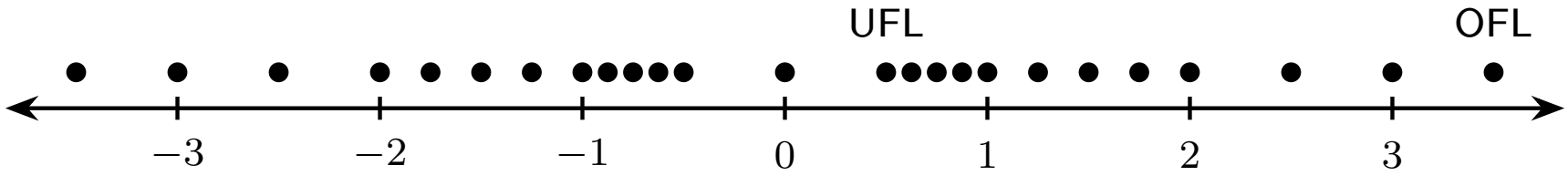
Most systems are **normalized**: $d_0 \neq 0$. In base 2 that forces $d_0 = 1$, so the leading bit is known in advance and need not be stored, a free extra digit of precision. Normalization also makes the representation **unique**: without it, 0.1×2^5 and 1.0×2^4 would be two spellings of the same value.

A Toy System You Can Write Out in Full

Take $\beta = 2$, $p = 3$, $L = -1$, $U = 1$. A normalized mantissa is $(1.d_1 d_2)_2$, so there are four of them, and three exponents. Every number in the system is one of these times ± 1:

	$1.00_2 = 1$	$1.01_2 = 1.25$	$1.10_2 = 1.5$	$1.11_2 = 1.75$
$E = -1, \quad \times 2^{-1}$	± 0.5	± 0.625	± 0.75	± 0.875
$E = 0, \quad \times 2^{0}$	± 1	± 1.25	± 1.5	± 1.75
$E = 1, \quad \times 2^{1}$	± 2	± 2.5	± 3	± 3.5

Twelve positive values, their twelve negatives, and 0 (which normalized form cannot spell, so it gets its own special pattern): **25 numbers in all**. That is the entire system. Nothing else exists: 0.3 does not, 4 does not, 0.1 does not.



Real Systems: What the Bits Buy You

format	bits (s/e/m)	p, ≈digits	range	where you meet it
half	16 (1/5/10)	11, 3	$10^{-4.2}$ to $10^{4.8}$	GPUs, mixed-precision ML
bfloat16	16 (1/8/7)	8, 2	$10^{\pm 38}$	TPUs, ML accelerators
single	32 (1/8/23)	24, 7	$10^{\pm 38}$	graphics, NN weights, `np.float32`
double	**64 (1/11/52)**	**53, 16**	$\mathbf{10^{\pm 308}}$	**the default: Python, NumPy, R, MATLAB**
extended	80 (1/15/64)	64, 19	$10^{\pm 4932}$	Intel FPU internals, `np.longdouble`
quad	128 (1/15/112)	113, 34	$10^{\pm 4932}$	climate, astrophysics codes
IBM 704 (1954)	36 (1/8/27)	27, 8	$10^{\pm 38}$	first mass-produced FP hardware
Cray-1 (1976)	64 (1/15/48)	48, 14	$10^{\pm 2466}$	classic supercomputer, pre-IEEE

Nobody agreed on a format until IEEE 754 (1985), so the same program gave different answers on different machines. And the trend is not upward: machine learning now runs on 16-bit formats with 2 to 3 digits, trading precision for speed, which is exactly why this lecture's failures are back.

Four Numbers Worth Naming

The **underflow limit** UFL (smallest positive number), the **overflow limit** OFL (largest), **machine epsilon** ε (the gap just above 1), and the **count**, each read off the table by picking the mantissa and exponent that produce it:

$$\text{UFL} = \underbrace{1}_{\substack{\text{smallest mantissa}\\ 1.00\ldots0}} \cdot \underbrace{\beta^{L}}_{\substack{\text{smallest}\\ \text{exponent}}} = \beta^{L}$$

$$\text{OFL} = \underbrace{(\beta - \beta^{1-p})}_{\substack{\text{largest mantissa}\\ \text{every digit } \beta-1}} \cdot \underbrace{\beta^{U}}_{\substack{\text{largest}\\ \text{exponent}}} = \beta^{U+1}\left(1-\beta^{-p}\right)$$

$$\varepsilon = \underbrace{\beta^{-(p-1)}}_{\substack{\text{one unit in the}\\ \text{last mantissa digit}}} \cdot \underbrace{\beta^{0}}_{\substack{\text{exponent}\\ \text{of } 1}} = \beta^{1-p}$$

$$\text{count} = \underbrace{2}_{\text{signs}} \cdot \underbrace{(\beta-1)}_{\substack{\text{choices}\\ \text{for } d_0\neq 0}} \cdot \underbrace{\beta^{\,p-1}}_{\substack{\text{choices for}\\ d_1,\ldots,d_{p-1}}} \cdot \underbrace{(U-L+1)}_{\text{exponents}} + \underbrace{1}_{\text{zero}}$$

Toy check ($\beta = 2$, $p = 3$, $L = -1$, $U = 1$): UFL $= 0.5$, OFL $= 1.75 \cdot 2 = 3.5$, $\varepsilon = 0.25$, count $= 2 \cdot 1 \cdot 4 \cdot 3 + 1 = 25$. All four match the table.

Machine Epsilon

Definition

Machine epsilon $\varepsilon = \beta^{1-p}$ is the **gap** between 1 and the next representable number: the smallest ε with $\mathrm{fl}(1+\varepsilon) > 1$.

- Toy system: $\varepsilon = 2^{1-3} = 0.25$, exactly the gap from 1 to 1.25 on the number line.
- Double ($p = 53$): $\varepsilon = 2^{-52} \approx 2.22 \times 10^{-16}$, or `np.finfo(float).eps`.

It is not really about 1: it is a relative step, valid at every size

The gap just above any $x = m \cdot \beta^E$ is $\varepsilon \cdot \beta^E$, so gaps **scale with the number**: in the toy system 0.125 near 0.5, 0.25 near 1, 0.5 near 2, always about $\varepsilon \cdot x$. So ε is **the smallest relative amount you can add to a number and have it change**: $x + y$ moves off x only when $|y| \gtrsim |x| \cdot \varepsilon/2$, whether x is 1 or 10^{20}.

Where It Bites, and Why It Usually Does Not

Absorption: The Small Number Disappears

A small number added to a large one

`>>> 1e20 + 1e-20 == 1e20` → `True`. The true sum, $100000000000000000000.00000000000000000001$, needs 40 significant digits; a double keeps 16, so everything past the sixteenth is rounded away and the larger number survives unchanged.

Adding y to x changes nothing whenever $|y| < |x| \cdot \varepsilon/2$: the small term falls inside the gap around the large one, so the nearest representable number is still x.

The Harmonic Series

Now run the harmonic series from Unit 1-4 in a loop, adding one term at a time to a running total S_n:

$$S_n = \sum_{k=1}^{n} \frac{1}{k} = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} \qquad \text{diverges: } S_n \to \infty.$$

On a computer the terms shrink while the total grows, so eventually $\frac{1}{n} < S_n \cdot \varepsilon/2$. From then on every term is **absorbed**, $S_{n+1} = S_n$ exactly, and the partial sums **freeze**. The divergent series *looks* convergent, and no error is raised.

Overflow and Underflow

Overflow: past OFL

```
>>> np.exp(1000)
inf
```

About 10^{434}, past the largest double. Everything computed from it is now `inf` or `nan`.

Underflow: below UFL

```
>>> np.exp(-1000)
0.0
```

A tiny positive number becomes exactly zero, and then $\ln(0) = -\infty$, or a division by it explodes.

The classic fix: work in logs

Multiplying 1000 probabilities underflows to 0. Taking logarithms turns the product into a sum (Unit 1-4) and the magnitudes back into something a double can hold:

$$\prod_{i=1}^{n} p_i \quad \longrightarrow \quad \sum_{i=1}^{n} \ln p_i$$

That is why statistics maximizes the **log**-likelihood, and why classifiers expose `predict_log_proba`, a numerical decision, not a mathematical one.

Cancellation: Subtracting Two Close Numbers

Subtracting nearly equal numbers is the most destructive operation in numerical computing. Watch it on a decimal machine that keeps $p = 4$ significant digits:

	true value	stored (4 digits)	relative error
a	1.23456	1.235	0.04%
b	1.23444	1.234	0.04%
$a - b$	0.00012	0.001	730%

Where the digits went

Both inputs were stored as well as a 4-digit machine can store anything. But their leading digits 1.23 were identical, so they cancelled to zero, and cancelling carries no information. What is left is the last digit of each, exactly where the rounding error lived. The error did not grow; the **result** shrank, so the error became a much larger *share* of it. A 16-digit double does the same, just further to the right.

This is called **catastrophic cancellation**, and it is a property of the *formula*, not of the data. Rearranging the algebra usually avoids it.

Keep It in Proportion

The double is better than your data

A double carries about 16 significant digits. A lab assay is good to 3, a survey answer to 1 or 2, a clinical measurement rarely to 4. For ordinary arithmetic on ordinary data, rounding error sits **ten trillion times below** the noise you already have. The **data**, not the arithmetic, is your accuracy bottleneck.

Every failure in this lecture was an *algorithm* amplifying that tiny error, and only a few shapes do it. Spot the shape, apply the habit, and stop worrying:

the shape	**the habit**
a sum spanning many magnitudes	`math.fsum` or `np.sum`, small terms first
a long product	work in logs: `logsumexp`
subtracting nearly equal numbers	rearrange, or use the library (`np.var`)
exact quantities: money, counts	integers, not floats

If none of those shapes is in your code, floating point is not your problem. The aim is to be **calm**, not careful: check for four things, then trust the machine.

Review

Review

What we covered

1. Memory is finite, so only finitely many reals are representable; every other one is **rounded** to a neighbor.
2. **Binary** places are powers of 2, and a **floating-point system** (β, p, L, U) is scientific notation in base β. A double keeps $p = 53$ bits, about 16 digits, over roughly $10^{\pm 308}$.
3. **UFL** $= \beta^L$ and **OFL** $= \beta^{U+1}(1 - \beta^{-p})$ are the edges; past them live `inf` and `nan`, and `nan != nan`. Spacing **doubles with each exponent**, so absolute error grows with magnitude, relative error does not.
4. A fraction is exact in binary only if its denominator is a power of 2, so 0.1 is stored a hair off, which is all that `0.1 + 0.2 != 0.3` means.
5. **Machine epsilon** $\varepsilon = \beta^{1-p} = 2^{-52}$ is the gap above 1; relative rounding error is at most $\varepsilon/2$.
6. Failures to recognize: **overflow** and **underflow** at the edges, **absorption** (a small term added to a large one vanishes, so a divergent sum can freeze) and **cancellation** (near-equal numbers subtracted lose digits). All are fixed by rearranging the computation, not by adding digits.
7. The habits: avoid subtracting near-equal numbers, work in logs for long products, use library sums and variances, and keep exact data (money, counts) in integers.

Notebook: `U1-5_FloatingPoint-1_MachineEpsilon`. **Previous:** Unit 1-4, Series. **Next:** Unit 2, **Linear Algebra**.